Hell's Kitchen Delivery no Google Cloud

Microsserviços em contêineres no Cloud Run, banco no Cloud SQL e tolerância a falhas

INTEGRANTES
Matheus Maia, Kelvin Bruno e Victor Hugo Ferreira

CÓDIGO-FONTE
github.com/matheus14maia/sd-2026-2

ENTREGA
02/12/2026

1 Contextualização e motivação

O Hell's Kitchen Delivery é o sistema de pedidos de um restaurante, com dois papéis que enxergam o mesmo dado por lados diferentes. O **cliente** cria a conta, monta a sacola a partir do cardápio (escolhendo, por exemplo, o sabor do refrigerante ou a água com ou sem gás), informa a entrega e acompanha o pedido em tempo real. A **cozinha** mantém o cardápio (pratos, descrições, fotos, preços e disponibilidade do dia, inclusive de cada opção), aceita os pedidos novos, avança as etapas até a entrega e vê em destaque os atrasados. Toda escolha oferecida ao cliente tem um controle correspondente na cozinha.

O domínio se divide em três contextos com dados e ritmos de uso próprios: **Usuários** (contas, credenciais e dados de entrega), **Catálogo** (o cardápio, lido por todos os clientes a cada 30 s e alterado pela cozinha) e **Pedidos** (escrita transacional, histórico de etapas e eventos em tempo real). Cada contexto é um microsserviço gRPC dono das próprias tabelas, sem chave estrangeira entre serviços; o pedido guarda uma cópia dos itens comprados. À frente deles, um **API Gateway** é a única porta de entrada do navegador: valida o JWT e o corpo da requisição, traduz JSON para Protobuf e concentra a política de tolerância a falhas.

No Trabalho 2 esse sistema rodava em três VMs do Compute Engine, e o grupo administrava sistema operacional, Docker, firewall, IPs fixos e a lista de IPs autorizados no banco, pagando pelas máquinas 24 horas por dia mesmo sem uso. A motivação desta etapa é entregar essas responsabilidades à plataforma. Cada componente virou uma imagem de contêiner publicada no **Artifact Registry** e implantada no **Cloud Run**, que fornece HTTPS com certificado automático, escala de zero a duas instâncias por serviço, substituição de instâncias com falha e logs centralizados. O PostgreSQL continua no **Cloud SQL**, agora acessado pelo conector do Cloud Run com autorização via IAM, e as senhas saíram dos arquivos `.env` para o **Secret Manager**. O resultado é uma infraestrutura 100% gerenciada, sem VM ligada, que fora do horário de uso custa centavos: o Cloud SQL é desligado entre os testes por um script.

2 Arquitetura no Google Cloud

A Figura 1 mostra os componentes implantados no projeto `sistemas-distribuidos-505422`, região `us-central1`, e a Tabela 1 resume o papel de cada serviço gerenciado. O caminho de um pedido (`POST /pedidos`) atravessa todas as camadas:

1. O navegador baixa a interface do `delivery-frontend`; o arquivo `/config.js`, gerado pelo nginx a partir da variável `API_URL`, informa o endereço do Gateway.
2. A chamada de dados vai direto ao `delivery-gateway` com `Authorization: Bearer`. O Gateway barra na borda o que é inválido (401 sem token válido, 403 sem o papel exigido, 400 para corpo fora do contrato) antes de qualquer chamada gRPC, e tira o dono do pedido do próprio token.
3. O Gateway monta o `CriarPedidoRequest` (Protobuf) e chama o `delivery-pedidos` por gRPC sobre HTTP/2 com TLS (porta 443), anexando um ID token da sua conta de serviço. O Cloud Run só entrega a chamada se essa conta tiver `roles/run.invoker` no serviço.
4. O Pedidos confere preço, disponibilidade e opção de cada item no Catálogo (`ConsultarItens`) com o prazo que resta da chamada original e grava pedido, itens e primeira etapa numa única transação no Cloud SQL; um `pg_notify` na mesma transação alimenta o tempo real.
5. A resposta volta como JSON (201). O id da requisição (`X-Requisicao-Id`) é o trace id e aparece em todas as linhas de log do caminho.

Endpoints públicos e privados. Os cinco serviços têm endpoint HTTPS público gerenciado pelo Cloud Run (domínio `run.app`, certificado automático). Frontend e Gateway aceitam qualquer cliente; os três microsserviços exigem o ID token de uma conta de serviço autorizada e respondem 403 a quem não o apresenta. Assim continua valendo a regra do Trabalho 2: o navegador só fala com o Gateway, e ninguém chega ao gRPC sem passar pela validação do JWT.

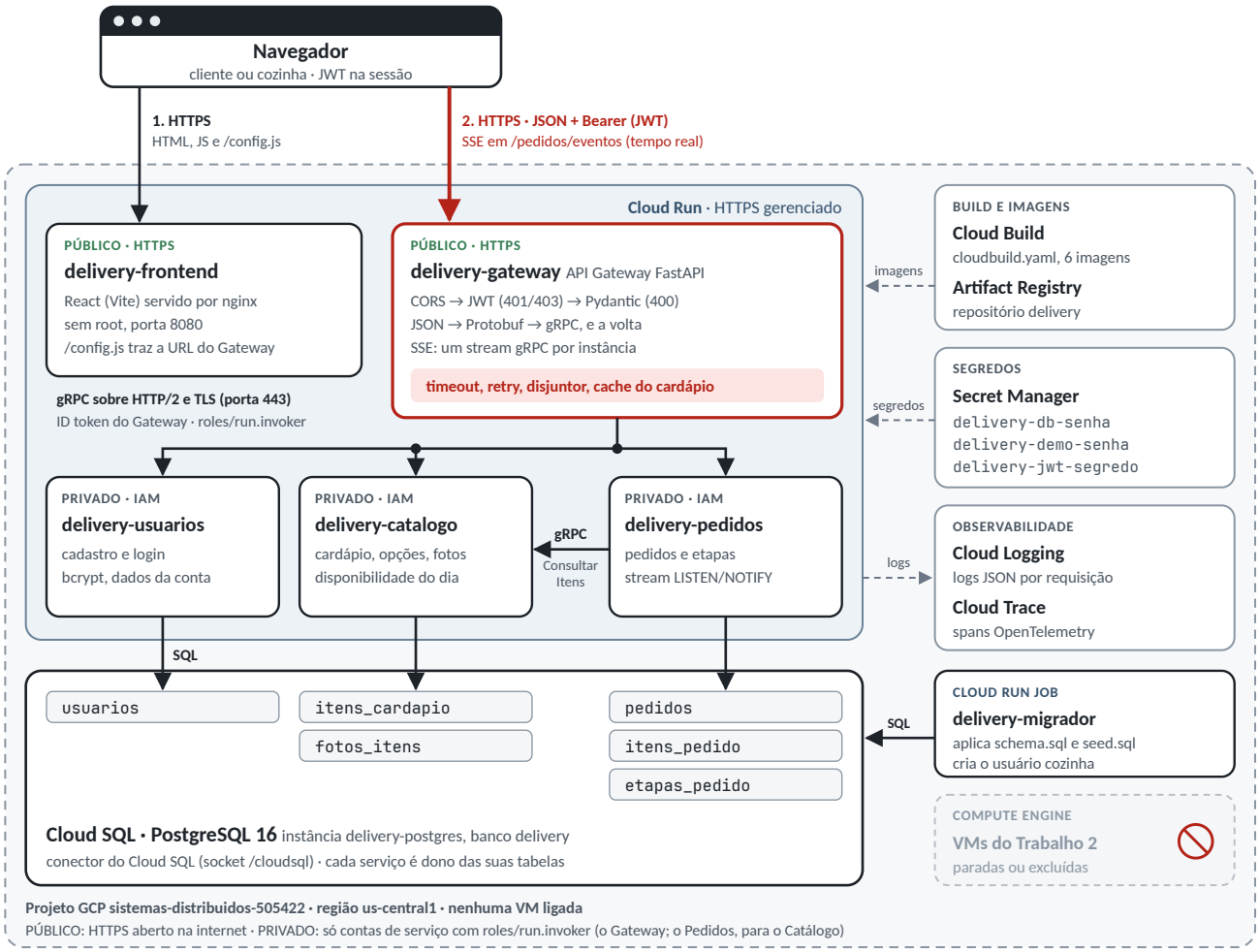


Figura 1. Arquitetura implantada no GCP. A seta vermelha é o caminho de uma requisição de dados do navegador; as setas tracejadas ligam os serviços de apoio (imagens, segredos e telemetria) ao Cloud Run. Fonte editável em `docs/arquitetura-cloud-run.svg`.

Tabela 1. Serviços gerenciados do GCP utilizados.

Serviço	Recursos	Uso no sistema
Cloud Run (serviços)	`delivery-frontend`, `delivery-gateway`, `delivery-usuarios`, `delivery-catalogo`, `delivery-pedidos`	Executa os contêineres com HTTPS gerenciado, de 0 a 2 instâncias por serviço e CPU extra na partida dos serviços Python (`--cpu-boost`); nos microsserviços, HTTP/2 de ponta a ponta para o gRPC (`--use-http2`)
Cloud Run (job)	`delivery-migrador`	Aplica `db/schema.sql` e `db/seed.sql` (idempotentes) e cria o usuário da cozinha a cada implantação
Cloud SQL	`delivery-postgres` (PostgreSQL 16, `db-f1-micro`), banco `delivery`	Persistência real dos dados; conexão pelo conector do Cloud Run (socket `/cloudsql`), com o papel `roles/cloudsql.client` e sem IP autorizado
Artifact Registry	repositório Docker `delivery` (`us-central1`)	Guarda as seis imagens, etiquetadas com o commit implantado
Cloud Build	`cloudbuild.yaml`	Constrói as seis imagens em paralelo (BuildKit) e publica no Artifact Registry
Secret Manager	`delivery-db-senha`, `delivery-demo-senha`, `delivery-jwt-segredo`	Segredos entregues ao contêiner como variáveis de ambiente (`--set-secrets`); cada conta de serviço lê só os seus
IAM	contas `delivery-gateway`, `delivery-servicos`, `delivery-frontend`	Menor privilégio por papel; `roles/run.invoker` define quem pode chamar cada microsserviço
Logging e Trace	Cloud Logging, Cloud Trace	Logs em JSON com o id da requisição; spans OpenTelemetry do Gateway até o banco, um por tentativa de retry
Compute Engine	VMs do Trabalho 2	Paradas ou excluídas por `desligar.sh`; `status.sh` lista as instâncias para comprovar

3 Tecnologias e frameworks

Tabela 2. Tecnologias por camada.

Camada	Tecnologias	Decisões principais
Frontend	React 18.3, Vite 5.4, nginx-unprivileged 1.28 (Alpine)	SPA sem biblioteca de componentes; `api.js` é o único módulo que fala com o backend; tempo real por SSE lido com `fetch`; URL do Gateway injetada na partida (`/config.js`), então a mesma imagem serve qualquer ambiente
API Gateway	Python 3.12, FastAPI 0.141, Pydantic 2.13, Uvicorn 0.53, PyJWT 2.10, grpcio 1.68	Middleware JWT (HS256) e checagem de papel; modelos com `extra="forbid"`; erros gRPC traduzidos para HTTP (indisponibilidade vira 503 com `Retry-After`); rotas finas, sem acesso ao banco
Microsserviços	Python 3.12, gRPC 1.68, Protocol Buffers 5.29, `grpc.health.v1`, reflection	Contratos em `proto/*.proto` como fonte da verdade (stubs gerados no build); `servico.py` com as regras e `repositorio.py` como único lugar com SQL; interceptador comum para trace, injeção de falha e erro de banco
Banco de dados	PostgreSQL 16 (Cloud SQL), psycopg 3.3 com pool, bcrypt	SQL puro, sem ORM; scripts idempotentes; fotos em `BYTEA`; `LISTEN/NOTIFY` para eventos; exclusão lógica; código de prato gerado sob `pg_advisory_xact_lock`
Observabilidade	OpenTelemetry 1.45 (exportador do Cloud Trace), logs JSON	O trace id é o id da requisição e viaja nos metadados gRPC; no ambiente local, Jaeger
Implantação	Docker multi-stage, Cloud Build, `gcloud` em scripts Bash, docker compose	`implantar.sh` (nove etapas, pode rodar de novo), `ligar.sh`, `desligar.sh`, `status.sh`; `SIMULAR=1` mostra os comandos sem executar

Dockerfiles e imagens

Há um Dockerfile por papel: `frontend/Dockerfile`, `docker/gateway.Dockerfile`, `docker/migrador.Dockerfile` e `docker/microsservico.Dockerfile`, que gera Usuários, Catálogo e Pedidos pelo argumento `SERVICO`. Todos são multi-stage. No Gateway e nos microsserviços, um estágio compila os `.proto` (o `grpcio-tools` fica fora da imagem final), outro instala as dependências num venv com cache do BuildKit e remove o `pip`, e o final copia só o venv, os stubs e o código do componente, já compilado em bytecode; o migrador segue o mesmo padrão sem o estágio dos stubs, e no frontend o Node existe só no estágio de build: a imagem final é o nginx com os arquivos estáticos. Todas rodam sem root; os serviços têm `HEALTHCHECK` e recebem o `SIGTERM` no processo principal, para encerrar de forma limpa; e o `.dockerignore` funciona como lista de permissão. As dependências são separadas por papel: o Gateway não leva o driver do PostgreSQL e o migrador não leva gRPC.

Tabela 3. Imagens publicadas no Artifact Registry (tamanho descompactado, `docker images`).

Imagem	Base do estágio final	Tamanho
`frontend`	`nginx-unprivileged:1.28-alpine`	81,8 MB (o build do Vite soma menos de 1 MB à base)
`gateway`	`python:3.12-slim`	276 MB
`usuarios`, `catalogo`, `pedidos`	`python:3.12-slim`	281 MB cada; as três compartilham as camadas de base e de dependências
`migrador`	`python:3.12-slim`	208 MB
referência: imagem única do Trabalho 2	`python:3.12-slim`, um estágio, root	308 MB, com o compilador de Protobuf e todas as dependências em todos os papéis

4 Estratégia de resiliência

Na nuvem, falha parcial é rotina: instância que sobe a frio, revisão nova substituindo a anterior, banco reiniciando, oscilação de rede. A estratégia combina recursos da plataforma com mecanismos no código, concentrados na classe `ClienteResiliente` (`src/comum/resiliencia.py`), usada pelo Gateway para chamar os três microsserviços e pelo Pedidos para chamar o Catálogo, e no interceptador comum dos servidores gRPC (`src/comum/servidor_grpc.py`). Todos os parâmetros vêm de variáveis de ambiente com valor padrão.

Tabela 4. Mecanismos de tolerância a falhas.

Mecanismo	Funcionamento	Configuração no Cloud Run
Timeout e prazo total	Cada tentativa tem um limite e a soma das tentativas também: nenhuma requisição fica pendurada esperando um serviço lento.	`GRPC_TIMEOUT=10` s e `GRPC_PRAZO_TOTAL=25` s (5 e 12 s no ambiente local; a folga cobre a partida a frio)
Retry com backoff	Até 3 tentativas, com esperas de ~0,2 s e ~0,4 s sorteadas entre 50% e 100% (jitter) para os clientes não repetirem juntos. Só repete o que é seguro: `UNAVAILABLE` sempre, pois ocorre antes de qualquer escrita; `DEADLINE_EXCEEDED` e `CANCELLED` só em operação idempotente.	`RETRY_TENTATIVAS=3`
Idempotência	O Gateway gera o `pedido_id` antes da primeira tentativa; repetir `CriarPedido` devolve o pedido já gravado (`ON CONFLICT DO NOTHING`), sem duplicar. Cadastro de prato e mudança de etapa também podem ser repetidos.	chave criada no Gateway
Circuit breaker	Um circuito por serviço. Após 3 chamadas seguidas que falharam mesmo com retry, ele abre e recusa na hora (503 com `Retry-After`); depois de 15 s libera uma chamada de teste (meio-aberto) e fecha se ela passar. O Catálogo fora não abre o circuito de Usuários nem de Pedidos.	`DISJUNTOR_FALHAS=3`, `DISJUNTOR_ABERTO_SEGUNDOS=15`
Fallback do cardápio	A última lista boa de cada categoria fica em memória no Gateway e é servida, com `"origem": "cache"` e um aviso, quando o Catálogo não responde. O cache só exibe o cardápio: pedido é sempre conferido no banco.	`CACHE_CARDAPIO_SEGUNDOS=1800`
Prazo propagado	O Pedidos chama o Catálogo com o tempo que resta da chamada do Gateway e, se o Catálogo falhar, avisa no metadado `x-falha-dependencia`: o Gateway não repete o `CriarPedido` (evita tempestade de retries) nem conta a falha contra o Pedidos.	automático
Banco indisponível	O interceptador converte erro de conexão com o PostgreSQL em `UNAVAILABLE`, que o cliente pode repetir, e o pool testa cada conexão antes de usar, reabrindo sozinho quando o Cloud SQL volta.	`DB_POOL_MAX=3` por instância
Tempo real	Se o stream gRPC `AcompanharPedidos` cair, o Gateway reabre com espera de 1 a 15 s e os navegadores veem "Reconectando..."; o `LISTEN` do PostgreSQL também reconecta e a tela segue consultando periodicamente.	`--timeout 3600` no Gateway e no Pedidos
Plataforma	O Cloud Run substitui instâncias com falha e só manda tráfego a uma revisão nova depois que ela sobe; no `SIGTERM` o serviço marca o health check como `NOT_SERVING` e termina as chamadas em andamento.	`--min-instances 0`, `--max-instances 2`, `--cpu-boost`

O que o usuário vê. Indisponibilidade nunca vira erro 500 genérico: `UNAVAILABLE`, `DEADLINE_EXCEEDED`, `CANCELLED` e `RESOURCE_EXHAUSTED` viram 503 com `Retry-After` e uma mensagem que o frontend apresenta em linguagem clara; o cardápio entra em modo de segurança com a lista salva; e a aba Saúde da cozinha mostra o health check de cada microsserviço e o estado de cada circuito.

Tabela 5. Roteiro automático `scripts/testar_resiliencia.py`, medido com docker compose derrubando o contêiner do Catálogo.

Situação	Resultado medido
Catálogo fora: `GET /cardapio`	200 com a lista salva (`origem: cache`) em 2,4 s, depois de esgotar o retry
Catálogo fora: `POST /pedidos`	503 em 0,5 s com `Retry-After: 5`; nada gravado
Isolamento: `GET /pedidos`	200 normal: Pedidos e Usuários continuam no ar
Depois de 3 falhas seguidas	circuito do Catálogo aberto; 503 em 3 ms, sem chamar o serviço
Catálogo religado	em ~16 s o circuito fecha, o cardápio volta a vir do Catálogo e `POST /pedidos` responde 201

Demonstração no Cloud Run. A falha é provocada pelo painel do GCP, sem mudar código: em *Cloud Run › delivery-catalogo › Editar e implantar nova revisão › Variáveis e secrets*, define-se `FALHA_PERCENTUAL=100`, e o interceptador passa a responder `UNAVAILABLE` como um serviço fora do ar. Ao vivo se observa o cardápio servido do cache, o pedido recusado com 503, o circuito aberto na aba Saúde e, no Cloud Logging, cada tentativa do retry (`repetindo em ... ms`) com o id da requisição. Com `FALHA_PERCENTUAL=50` o retry absorve as falhas sem o usuário perceber; com `FALHA_ATRASO_MS=12000` é o timeout que corta a espera. Voltando para 0, tudo se recupera em cerca de 15 s. Os 84 cenários de `scripts/testar_gateway.py` (status HTTP, autenticação e regras de negócio) passaram contra a URL pública do Gateway em 11/11/2026, e o roteiro da Tabela 5 também roda contra a nuvem (`testar_resiliencia.py --url`, com os comandos `gcloud run services update` de derrubar e religar).

5 Links públicos HTTPS e configuração no Cloud Run

Tabela 6. Serviços implantados em 11/11/2026 no Cloud Run (região `us-central1`). A tabela é impressa no fim de `scripts/gcp/cloud-run/implantar.sh` e por `status.sh`.

Serviço	Acesso	URL
`delivery-frontend` interface do cliente e da cozinha	**público**	`https://delivery-frontend-j76gmjruwa-uc.a.run.app`
`delivery-gateway` API HTTP/JSON, documentação em `/docs`	**público**	`https://delivery-gateway-j76gmjruwa-uc.a.run.app`
`delivery-usuarios` gRPC: contas e login	**IAM: Gateway**	`https://delivery-usuarios-j76gmjruwa-uc.a.run.app`
`delivery-catalogo` gRPC: cardápio, opções e fotos	**IAM: Gateway e Pedidos**	`https://delivery-catalogo-j76gmjruwa-uc.a.run.app`
`delivery-pedidos` gRPC: pedidos e tempo real	**IAM: Gateway**	`https://delivery-pedidos-j76gmjruwa-uc.a.run.app`

Implantação. Tudo é implantado pelo Cloud Shell com um comando, `bash scripts/gcp/cloud-run/implantar.sh`, que pode ser repetido a cada versão: habilita as APIs, cria o repositório de imagens, liga o Cloud SQL, cria os segredos (senhas sorteadas; só a da cozinha é digitada) e as contas de serviço com papéis mínimos, constrói as imagens no Cloud Build, executa o job de migração e implanta os microsserviços e, por fim, Gateway e frontend, com o CORS apontando para a URL do frontend. Entre os testes, `desligar.sh` desliga o Cloud SQL, o maior custo, e para ou exclui as VMs do Trabalho 2; `ligar.sh` religa o banco antes da apresentação; o Cloud Run, com mínimo de zero instâncias, não cobra parado.

Tabela 7. Variáveis de ambiente de cada serviço, visíveis e editáveis no painel do Cloud Run (*Editar e implantar nova revisão › Variáveis e secrets*). Os segredos aparecem como referência ao Secret Manager, sem o valor.

Serviço	Variáveis de ambiente	Segredos
`delivery-gateway`	`USUARIOS_HOST`, `CATALOGO_HOST` e `PEDIDOS_HOST` (endereços `run.app`) com `*_PORTA=443`, que liga TLS e ID token; `CORS_ORIGENS` (URL do frontend); `GRPC_TIMEOUT` e `GRPC_PRAZO_TOTAL`; `LOG_JSON=1`; `RASTREAMENTO=gcp`	`JWT_SEGREDO`
`delivery-usuarios`, `delivery-catalogo`	`DB_HOST=/cloudsql/<conexão>`, `DB_NAME`, `DB_USER`, `DB_POOL_MAX=3`, `MAX_WORKERS=20`, tempos do gRPC, `LOG_JSON=1`, `RASTREAMENTO=gcp`; na demonstração, `FALHA_PERCENTUAL` e `FALHA_ATRASO_MS`	`DB_PASSWORD`
`delivery-pedidos`	as mesmas, mais `CATALOGO_HOST` e `CATALOGO_PORTA=443`	`DB_PASSWORD`
`delivery-frontend`	`API_URL` (URL do Gateway, servida em `/config.js`)	nenhum
`delivery-migrador` (job)	`DB_*` como acima e `DEMO_LOGIN=professor`	`DB_PASSWORD`, `DEMO_SENHA`